SISTEMAS EMBEBIDOS

Universidad Politécnica del

Bicentenario.

Sistema de Control Proporcional de Temperatura con ESP32, Accionamiento Mecánico por Servomotor y Dimmer de Potencia

Ingeniería Mecatrónica 701

Edgar Uriel Rodríguez Moreno *24033660*

Jorge Zamorano Chávez *24033247*

Axel Gabriel Domínguez Macías *24032952*

Braulio Vázquez Morado *24032975*

Daisy Jackeline Hernández Ramos *24033069*

Silao, Guanajuato a 6 de Noviembre de 2026.

Introducción

En la ingeniería el control automático y los sistemas embebidos, la regulación continua de variables físicas representa un salto cualitativo fundamental respecto a los esquemas convencionales de conmutación todo-nada. Mientras que un controlador ON-OFF únicamente alterna entre el encendido total y el apagado completo de un actuador generando oscilaciones térmicas constantes alrededor del punto de operación y desgaste mecánico en los elementos de conmutación, un controlador proporcional ajusta de manera gradual y continua la energía suministrada al proceso en función directa de la magnitud del error existente entre la temperatura deseada y la temperatura real medida. Esta estrategia permite suavizar la respuesta dinámica del sistema térmico, evitar cambios bruscos de potencia y aproximar la variable controlada hacia su valor de referencia con mayor estabilidad.

El presente reporte documenta el diseño electromecánico, la implementación electrónica, el desarrollo de firmware y la validación experimental de un sistema de control proporcional de temperatura gobernado por un microcontrolador ESP32. A diferencia de los reguladores comerciales que emplean etapas de control de fase optoacopladas directamente gobernadas por señales digitales, este prototipo implementa una solución mecatrónica híbrida sumamente didáctica y segura: la acción de control calculada porcentualmente por el algoritmo proporcional se traduce en una posición angular específica ejecutada por un servomotor SG90, cuyo eje se encuentra acoplado mecánicamente a la perilla de un dimmer regulador de potencia de corriente alterna. De este modo, el giro del servomotor modula físicamente la potencia eléctrica entregada a un elemento calefactor alojado en el interior de una cámara térmica, manteniendo un aislamiento galvánico total entre la electrónica digital de bajo voltaje y la red eléctrica de potencia. Asimismo, el sistema incorpora un sensor digital de temperatura DS18B20, una pantalla LCD de 16×2 con adaptador I2C para el monitoreo en tiempo real de la temperatura, el punto de referencia, el error, la salida porcentual y el ángulo del servomotor, así como un

teclado matricial 4×4 que permite reconfigurar dinámicamente tanto la temperatura de referencia como la ganancia proporcional sin necesidad de recompilar el código.

Marco Teórico

El desempeño del controlador proporcional desarrollado se fundamenta en la integración de diversos dispositivos electrónicos, transductores, actuadores electromecánicos y etapas de potencia, cuyas características operativas y compatibilidad de programación se detallan a continuación.

El *microcontrolador ESP32 DevKit V1* constituye el núcleo de procesamiento y cálculo matemático del sistema. Este System-on-Chip de 32 bits fabricado por Espressif Systems integra un procesador Xtensa LX6 de doble núcleo capaz de operar hasta a 240 MHz, memoria SRAM interna, periféricos de comunicación serie como I2C, SPI y UART, así como controladores dedicados de modulación por ancho de pulso LEDC de alta resolución ideales para el gobierno preciso de servomotores. Dentro de esta práctica, el ESP32 ejecuta de forma cíclica la adquisición de temperatura mediante protocolo 1-Wire, calcula el error térmico, evalúa la ley de control proporcional con saturación, convierte el porcentaje de salida en un ángulo calibrado de movimiento y gestiona simultáneamente la interfaz de usuario compuesta por la pantalla LCD y el teclado matricial. Respecto a su compatibilidad de programación, el ESP32 soporta C y C++ a través del framework Arduino y el entorno oficial ESP-IDF en plataformas como PlatformIO y Arduino IDE, además de ser compatible con MicroPython, CircuitPython, Lua, Rust y JavaScript.

El *sensor digital de temperatura DS18B20* actúa como el elemento primario de retroalimentación del lazo cerrado de control. Desarrollado por Maxim Integrated, actualmente Analog Devices, este termómetro digital mide temperaturas desde -55 °C hasta +125 °C con una precisión de ±0.5 °C y entrega la lectura directamente en formato digital binario a través del bus propietario 1-Wire utilizando una única línea

de datos asistida por una resistencia pull-up externa de 4.7 kΩ. Su inmunidad al ruido electromagnético resulta determinante en este montaje, ya que evita alteraciones en la medición causadas por la conmutación de potencia del dimmer de corriente alterna. En cuanto a lenguajes de programación, el DS18B20 es compatible con C y C++ mediante las librerías OneWire y DallasTemperature en Arduino y PlatformIO, MicroPython con los módulos ds18x20 y onewire, Python en sistemas Linux embebidos, C estándar en microcontroladores AVR o PIC, y Rust.

La *pantalla LCD alfanumérica de 16×2 con interfaz I2C* cumple la función de monitor local de variables de proceso y parámetros de sintonización. Basada en el controlador HD44780 acoplado a un expansor de puertos PCF8574, permite visualizar dos líneas de dieciséis caracteres empleando únicamente las líneas SDA y SCL del bus I2C, simplificando el cableado e incorporando ajuste de contraste y retroiluminación. En este proyecto despliega dos vistas intercambiables: una pantalla principal con la temperatura medida, el setpoint, el error y el porcentaje de salida, y una pantalla secundaria con la ganancia proporcional vigente y el ángulo exacto del servomotor. Este periférico es compatible con C y C++ mediante bibliotecas como LiquidCrystal_I2C en Arduino y PlatformIO o controladores I2C nativos en ESP-IDF, así como con MicroPython, Python, CircuitPython y ensamblador.

El *teclado matricial de membrana 4×4* proporciona la interfaz de entrada de datos para sintonizar el controlador en tiempo real. Estructurado como una matriz pasiva de cuatro filas por cuatro columnas de interruptores normalmente abiertos, permite al microcontrolador identificar cualquiera de sus dieciséis teclas mediante técnicas de barrido digital. En el sistema desarrollado, las teclas A y B habilitan respectivamente la edición de la temperatura de referencia y de la ganancia proporcional, las teclas numéricas y el asterisco * permiten ingresar valores enteros y decimales, la tecla C borra la entrada en curso, la tecla # confirma y valida el nuevo parámetro, y la tecla D alterna entre las pantallas de monitoreo o cancela una edición. Su programación es compatible con C y C++ utilizando la biblioteca Keypad

en Arduino y PlatformIO o rutinas matriciales en ESP-IDF, MicroPython, CircuitPython, Python y Rust.

El *servomotor micro SG90* desempeña el papel de actuador electromecánico de posicionamiento angular encargado de girar físicamente el eje del potenciómetro del dimmer. Este dispositivo integra un motor de corriente continua, una caja reductora de engranajes, un potenciómetro interno de retroalimentación de posición y un circuito controlador que interpreta señales periódicas moduladas por ancho de pulso a una frecuencia de 50 Hz. Al variar el ancho del pulso entre 500 y 2400 microsegundos, el eje del servomotor se desplaza con precisión hacia el ángulo comandado y mantiene el torque necesario para vencer la fricción mecánica de la perilla reguladora. Respecto a su compatibilidad de software, el servomotor SG90 puede controlarse mediante C y C++ empleando librerías especializadas como ESP32Servo o Servo en PlatformIO y Arduino, además de MicroPython, CircuitPython, Python en Raspberry Pi, Rust y lenguaje ensamblador.

El *módulo dimmer regulador de potencia AC* constituye el elemento final de control encargado de modular la energía eléctrica entregada al calefactor. Basado en un tiristor bidireccional o TRIAC gobernado por un circuito de disparo con DIAC, resistencia variable y capacitor, este módulo recorta el ángulo de conducción de cada semiciclo de la onda senoidal de corriente alterna conforme se gira su potenciómetro rotativo. Al variar la resistencia del potenciómetro mediante el acoplamiento mecánico con el servomotor, se modifica el instante de disparo del TRIAC y, por ende, el valor eficaz de voltaje y potencia aplicada a la carga resistiva. Debido a que su ajuste es puramente mecánico a través del eje del potenciómetro y está gobernado indirectamente por el servomotor, es compatible con cualquier lenguaje de programación capaz de generar señales PWM en el microcontrolador anfitrión, incluyendo C, C++, MicroPython, Python y Rust.

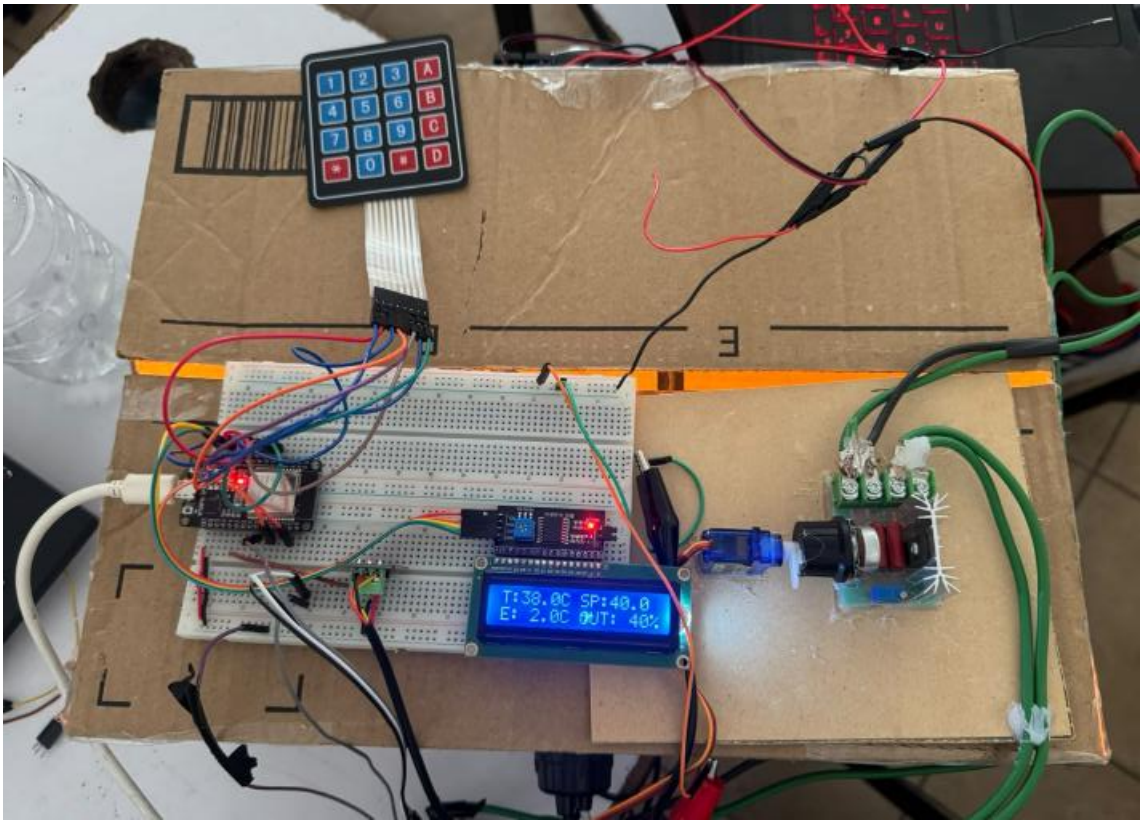
El *elemento calefactor y el sistema auxiliar de extracción térmica con celda Peltier* conforman la planta térmica bajo estudio dentro del recinto cerrado. El foco calefactor convierte la energía eléctrica modulada por el dimmer en calor por efecto

Joule, elevando la temperatura del aire interior de la cámara, mientras que el módulo termoeléctrico Peltier con disipador y ventilador permite disipar o extraer calor del recinto durante las pruebas de enfriamiento y perturbación térmica. Al operar como cargas eléctricas de potencia aisladas del microcontrolador, son compatibles con cualquier entorno y lenguaje de programación embebido como C, C++, MicroPython o Python.

Las *protoboards, cables jumper Dupont, caimanes de conexión, conectores de tornillo y fuente de alimentación externa de 5 V* aseguran la interconexión ordenada de los buses de datos, el suministro estable de corriente para el servomotor sin provocar caídas de tensión en el ESP32 y la unión de referencias de tierra común indispensables para el funcionamiento confiable del prototipo.

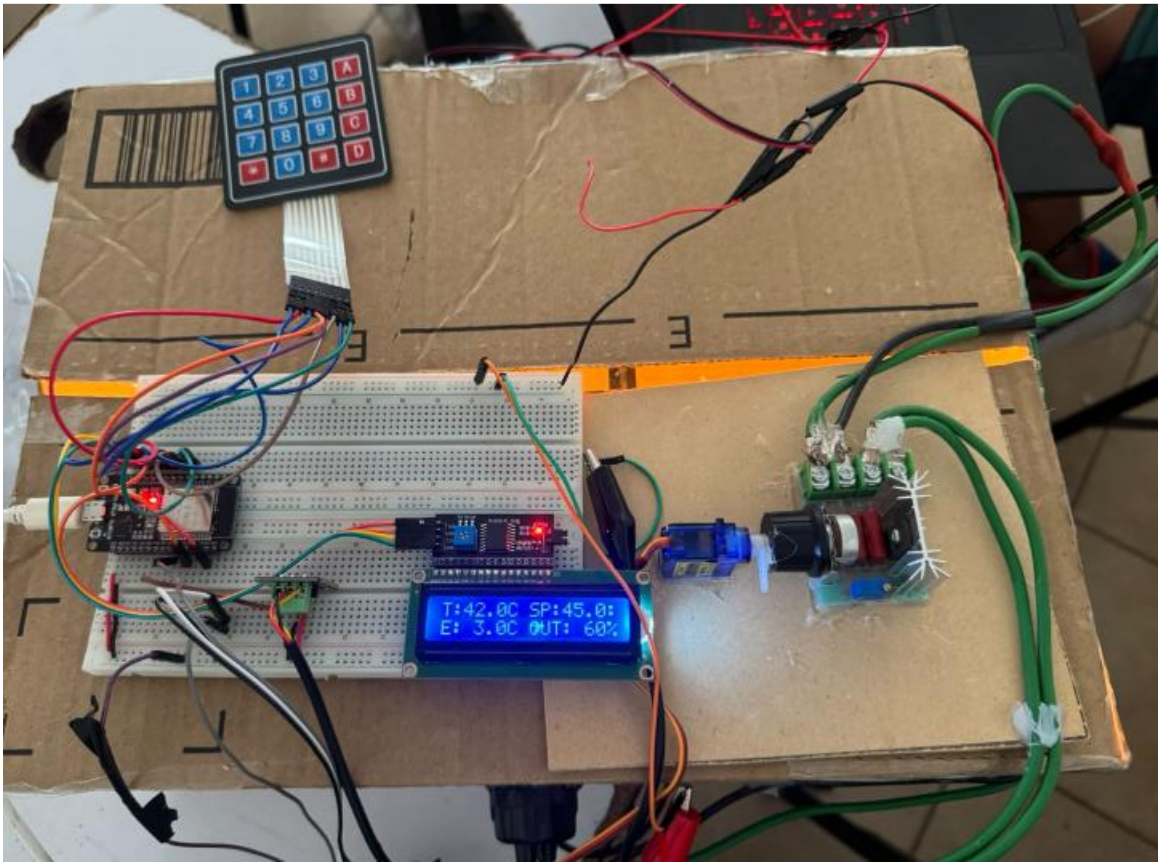
Desarrollo de la Práctica

La implementación del controlador proporcional partió del aprovechamiento y adaptación estructural de la cámara térmica construida previamente, acondicionando su parte superior y lateral para integrar los nuevos elementos electromecánicos requeridos por la práctica. En primer lugar, se realizó el acoplamiento mecánico entre el servomotor SG90 y el módulo dimmer de potencia. Para ello, se fijó firmemente tanto la placa del dimmer como el cuerpo del servomotor sobre una base rígida de cartón reforzado colocada en la cubierta superior de la cámara térmica, alineando coaxialmente el eje rotativo del servomotor con la perilla del potenciómetro del dimmer y uniéndolos solidariamente mediante adhesivo termofusible y un acople plástico rígido. Antes de energizar la carga de corriente alterna, se llevó a cabo una calibración experimental del recorrido mecánico del conjunto, determinando que debido a la zona muerta inicial del potenciómetro del dimmer, el elemento calefactor comenzaba a encender exactamente a partir de los 90 grados y alcanzaba su potencia máxima a los 180 grados; por consiguiente, se establecieron dichos valores como los límites ANGULO_MIN y ANGULO_MAX dentro del programa.



Circuito

Posteriormente, se ejecutó el cableado electrónico siguiendo estrictas medidas de aislamiento entre la etapa de baja tensión y los conductores verdes de potencia de corriente alterna conectados a las borneras del dimmer. En la protoboard principal se alojó la tarjeta ESP32 DevKit V1, conectando el bus I2C de la pantalla LCD a los pines GPIO 21 para SDA y GPIO 22 para SCL, y enlazando la línea de datos del sensor DS18B20 al pin GPIO 4 con su resistencia pull-up de 4.7 kΩ hacia 3.3 V. El teclado matricial 4×4 se conectó mediante cables Dupont a los pines GPIO 13, 12, 14 y 27 para las cuatro filas y GPIO 26, 25, 33 y 32 para las cuatro columnas, ubicándolo en el extremo izquierdo de la cubierta para facilitar la introducción de parámetros. Por su parte, la señal PWM del servomotor SG90 se conectó al pin GPIO 18, alimentando su bus de potencia desde una línea dedicada de 5 V con tierra común respecto al ESP32 para evitar reinicios por picos de corriente durante el giro de la perilla.

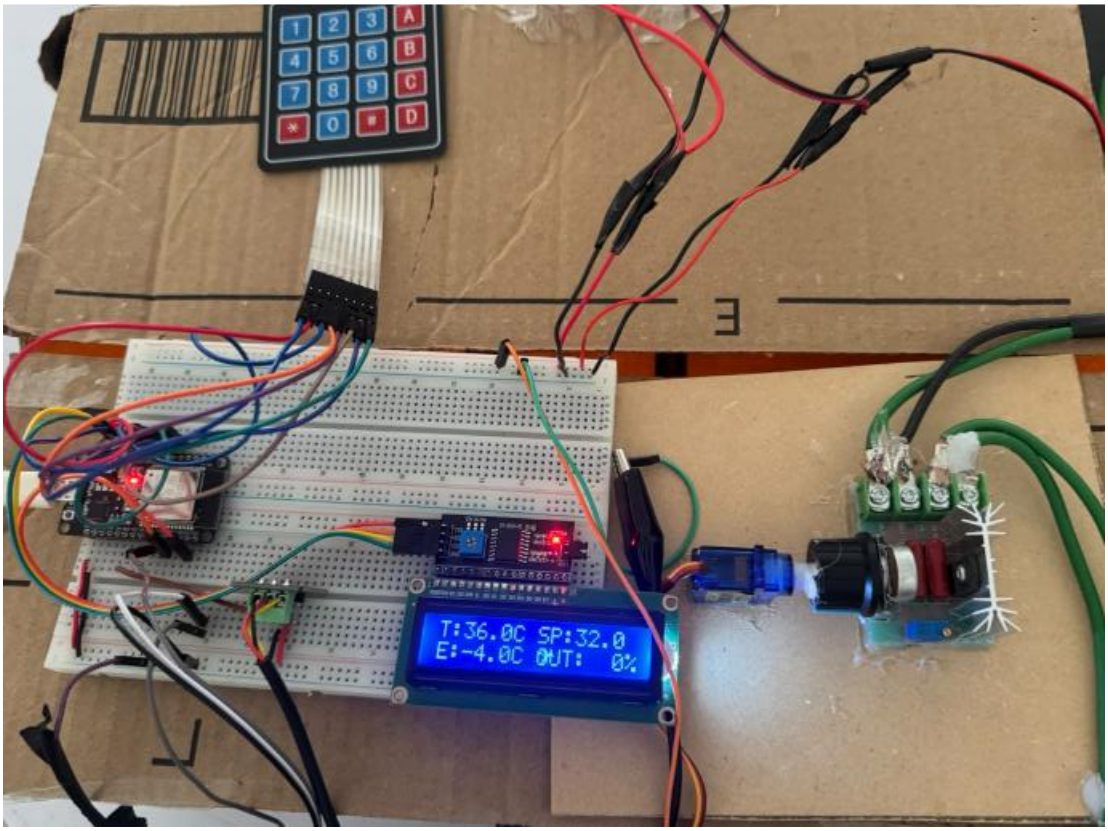


Incremento de la temperatura sin celda peltier

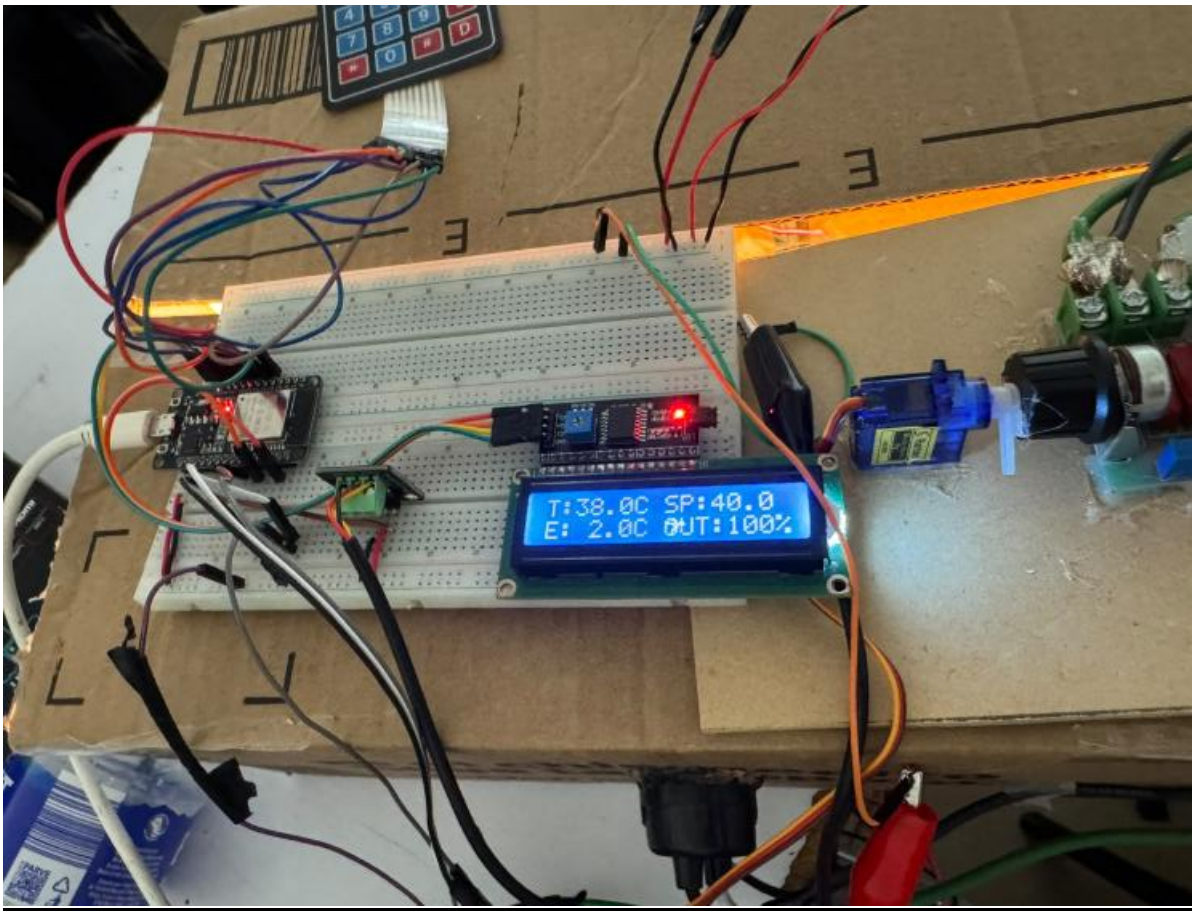
En el entorno de desarrollo Visual Studio Code con PlatformIO, se estructuró el firmware en lenguaje C++ empleando las librerías OneWire, DallasTemperature, LiquidCrystal_I2C, Keypad y ESP32Servo. Dentro de la función setup, se asignó un temporizador PWM dedicado de 50 Hz al servomotor mediante ESP32PWM::allocateTimer y se posicionó inicialmente en 90 grados para mantener apagado el calefactor al arranque. En el bucle principal loop, se implementó una arquitectura no bloqueante gobernada por millis que lee el teclado matricial en tiempo real y muestrea la temperatura del sensor DS18B20 cada 500 milisegundos.

Cada vez que se obtiene una nueva lectura térmica, la función calcularControlProporcional determina el error instantáneo restando la temperatura medida a la temperatura de referencia, multiplica dicho error por la ganancia proporcional Kp y aplica una etapa de saturación para acotar la acción de control

entre el 0 % y el 100 %. Acto seguido, realiza un mapeo lineal dentro del rango calibrado de 90 a 180 grados y comanda el desplazamiento angular del servomotor sobre el dimmer. De forma paralela, la rutina actualizarLCD refresca los indicadores en pantalla y permite alternar con la tecla D entre la vista de variables de proceso y la vista de ganancia proporcional y ángulo del servo, validando además cualquier modificación de T_ref o Kp introducida con las teclas A y B.



Decremento de la temperatura



Temperatura max del horno con celda peltier prendida

```
#include <Arduino.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <ESP32Servo.h>

#define DS18B20_PIN 4
#define SERVO_PIN   18
#define LCD_SDA     21
#define LCD_SCL     22
#define LCD_ADDR    0x27

// --- RE-CALIBRACIÓN MECÁNICA ---
// Si a los 90° apenas enciende, ese debe ser tu punto de
partida (0% de potencia).
const int ANGULO_MIN = 90;  // 0%  de control -> 90° (Umbral de
encendido)
const int ANGULO_MAX = 180; // 100% de control -> 180° (Potencia
máxima)
```

```
const byte FILAS = 4;
const byte COLUMNAS = 4;

char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

byte pinesFilas[FILAS]       = {13, 12, 14, 27};
byte pinesColumnas[COLUMNAS] = {26, 25, 33, 32};

Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas,
pinesColumnas, FILAS, COLUMNAS);


OneWire oneWire(DS18B20_PIN);
DallasTemperature sensors(&oneWire);
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Servo dimmerServo;

float T_medida = 0.0;
float T_ref = 40.0;         // Setpoint inicial (°C)
float Kp = 5.0;             // Ganancia Proporcional (%/°C)
float errorTemp = 0.0;      // e(t) = T_ref - T_medida
float accionControl = 0.0;  // u(t) en porcentaje (0% a 100%)
int anguloServo = ANGULO_MIN;

enum ModoSistema { MODO_NORMAL, MODO_EDIT_SP, MODO_EDIT_KP };
ModoSistema modoActual = MODO_NORMAL;

String bufferTeclado = "";
bool vistaAlternativa = false;

unsigned long ultimaLecturaTemp = 0;
const unsigned long INTERVALO_TEMP = 500;

void calcularControlProporcional() {
  errorTemp = T_ref - T_medida;

  // Ley de control proporcional: u(t) = Kp * e(t)
  float u_calculado = Kp * errorTemp;

  // Saturación del controlador (0% a 100%)
  if (u_calculado < 0.0) {
    accionControl = 0.0;
  } else if (u_calculado > 100.0) {
    accionControl = 100.0;
  } else {
    accionControl = u_calculado;
  }

  // Mapeo lineal considerando la zona muerta (90° a 180°)
  // θ = 90 + (u / 100) * (180 - 90)
  float anguloFlotante = ANGULO_MIN + (accionControl / 100.0) *
(ANGULO_MAX - ANGULO_MIN);
```

```
  anguloServo = (int)constrain(anguloFlotante,
(float)ANGULO_MIN, (float)ANGULO_MAX);

  // Posicionar servomotor
  dimmerServo.write(anguloServo);
}

void actualizarLCD() {
  char linea0[17];
  char linea1[17];

  if (modoActual == MODO_EDIT_SP) {
    snprintf(linea0, sizeof(linea0), "Set T_ref (°C): ");
    snprintf(linea1, sizeof(linea1), "> %-14s",
bufferTeclado.c_str());
  }
  else if (modoActual == MODO_EDIT_KP) {
    snprintf(linea0, sizeof(linea0), "Set Kp (%%/°C):  ");
    snprintf(linea1, sizeof(linea1), "> %-14s",
bufferTeclado.c_str());
  }
  else {
    if (!vistaAlternativa) {
      snprintf(linea0, sizeof(linea0), "T:%4.1fC SP:%4.1f",
T_medida, T_ref);
      snprintf(linea1, sizeof(linea1), "E:%4.1fC OUT:%3d%%",
errorTemp, (int)accionControl);
    } else {
      snprintf(linea0, sizeof(linea0), "Kp: %4.1f %%/C   ", Kp);
      snprintf(linea1, sizeof(linea1), "SERVO: %3d DEG  ",
anguloServo);
    }
  }

  lcd.setCursor(0, 0);
  lcd.print(linea0);
  lcd.setCursor(0, 1);
  lcd.print(linea1);
}

void procesarTeclado(char tecla) {
  if (tecla == 'A') {
    modoActual = MODO_EDIT_SP;
    bufferTeclado = "";
  }
  else if (tecla == 'B') {
    modoActual = MODO_EDIT_KP;
    bufferTeclado = "";
  }
  else if (tecla == 'C') {
    bufferTeclado = "";
  }
  else if (tecla == 'D') {
    if (modoActual != MODO_NORMAL) {
      modoActual = MODO_NORMAL;
      bufferTeclado = "";
    } else {
      vistaAlternativa = !vistaAlternativa;
    }
```

```
  }
  else if (tecla == '#') {
    if (bufferTeclado.length() > 0) {
      float valor = bufferTeclado.toFloat();
      if (modoActual == MODO_EDIT_SP) {
        if (valor >= 0.0 && valor <= 100.0) T_ref = valor;
      } else if (modoActual == MODO_EDIT_KP) {
        if (valor >= 0.0 && valor <= 50.0) Kp = valor;
      }
    }
    modoActual = MODO_NORMAL;
    bufferTeclado = "";
  }
  else if ((tecla >= '0' && tecla <= '9') || tecla == '*') {
    if (modoActual != MODO_NORMAL) {
      char c = (tecla == '*') ? '.' : tecla;
      if (c == '.' && bufferTeclado.indexOf('.') != -1) return;
      if (bufferTeclado.length() < 6) bufferTeclado += c;
    }
  }
}

void setup() {
  Serial.begin(115200);

  ESP32PWM::allocateTimer(0);
  dimmerServo.setPeriodHertz(50);
  dimmerServo.attach(SERVO_PIN, 500, 2400);
  dimmerServo.write(ANGULO_MIN); // Inicia en 90° (foco apagado)

  Wire.begin(LCD_SDA, LCD_SCL);
  lcd.init();
  lcd.backlight();

  sensors.begin();
  sensors.setResolution(9);
  sensors.setWaitForConversion(false);
  sensors.requestTemperatures();

  lcd.setCursor(0, 0);
  lcd.print(" Control P Temp ");
  lcd.setCursor(0, 1);
  lcd.print(" ESP32 + Keypad ");
  delay(1500);
  lcd.clear();
}

void loop() {
  unsigned long ahora = millis();

  char tecla = teclado.getKey();
  if (tecla != NO_KEY) {
    procesarTeclado(tecla);
    actualizarLCD();
  }

  if (ahora - ultimaLecturaTemp >= INTERVALO_TEMP) {
    ultimaLecturaTemp = ahora;
```

```
    float tempRead = sensors.getTempCByIndex(0);
    if (tempRead > -55.0 && tempRead < 125.0) {
      T_medida = tempRead;
    }
    sensors.requestTemperatures();

    calcularControlProporcional();
    actualizarLCD();

    Serial.printf("T: %.2f C | SP: %.2f C | Error: %.2f C | OUT:
%.1f %% | Servo: %d deg\n",
                  T_medida, T_ref, errorTemp, accionControl,
anguloServo);
  }

                                  }
```

Diagrama Repositorio en Github

Resultados

Las pruebas experimentales realizadas sobre el prototipo físico validaron de manera contundente el correcto funcionamiento del controlador proporcional y de su mecanismo de actuación electromecánica. Durante la fase de calibración inicial, la compensación por software de la zona muerta del dimmer entre 90° y 180° permitió que todo el rango útil de control del 0 % al 100 % se tradujera en una variación efectiva y progresiva de la intensidad luminosa y térmica del foco calefactor, sin atascos mecánicos ni esfuerzos excesivos en el servomotor SG90.

En las pruebas de seguimiento de referencia y respuesta ante cambios de parámetros, al establecer un punto de referencia de 40.0 °C con el horno a 38.0 °C, el sistema calculó un error de 2.0 °C y posicionó el servomotor para entregar una salida proporcional del 40 %. Posteriormente, al incrementar la temperatura de referencia a 45.0 °C durante la prueba de calentamiento sin la celda Peltier activa, el sensor DS18B20 registró el ascenso térmico progresivo hasta alcanzar 42.0 °C, reduciendo proporcionalmente el error a 3.0 °C y modulando la salida al 60 % conforme la temperatura se aproximaba al objetivo.

Por el contrario, al disminuir repentinamente el setpoint a 32.0 °C cuando el recinto se encontraba a 36.0 °C, el controlador calculó un error negativo de -4.0 °C, activando correctamente la condición de saturación inferior al 0 % y regresando el servomotor a su posición de reposo de 90 grados para cortar por completo la alimentación al calefactor y permitir el descenso de la temperatura. Finalmente, al someter el horno a una perturbación térmica severa encendiendo la celda Peltier de enfriamiento mientras se solicitaba un setpoint de 40.0 °C, la extracción constante de calor mantuvo la temperatura interna alrededor de 38.0 °C, demostrando cómo el controlador incrementa su acción de salida hasta alcanzar la saturación máxima del 100 % para contrarrestar la carga térmica y reflejando claramente el fenómeno teórico del error en estado estable inherente a los controladores puramente proporcionales.

Conclusiones

La implementación de este sistema permitió comprobar de forma práctica y visual las diferencias fundamentales entre un control todo-nada y un control continuo de tipo proporcional aplicado a procesos térmicos. El acoplamiento mecánico entre un servomotor SG90 y un dimmer comercial demostró ser una alternativa segura, económica y didáctica para modular potencia eléctrica en corriente alterna sin comprometer el aislamiento galvánico del microcontrolador ESP32, siempre y cuando se realice una caracterización previa de la zona muerta del potenciómetro y se calibren los límites angulares en el algoritmo de mapeo.

Asimismo, el análisis experimental con y sin perturbación térmica mediante la celda Peltier evidenció con total claridad tanto la suavidad de regulación del controlador proporcional como su limitación característica frente a perturbaciones sostenidas, donde la ausencia de una acción integral impide eliminar por completo el error en estado estable a menos que se incremente considerablemente la ganancia proporcional. En conjunto, la integración del sensor DS18B20, el teclado matricial 4×4, la pantalla LCD I2C y la programación no bloqueante en PlatformIO cumplió satisfactoriamente con todos los objetivos académicos de la práctica.

Referencias

Analog Devices. *DS18B20: Programmable Resolution 1-Wire Digital Thermometer Datasheet*. Analog Devices, Inc., 2019.

Bolton, W. *Ingeniería de control*. Alfaomega Grupo Editor, 2013.

Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems Co., Ltd., 2023.

Monk, S. *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education, 2022.

Ogata, K. *Ingeniería de control moderna*. Pearson Educación, 2010.

PlatformIO. *PlatformIO Documentation: Espressif 32 Platform and Arduino Framework*. PlatformIO Labs, 2024.

Tower Pro. *SG90 9g Micro Servo Technical Specification Sheet*. Tower Pro Pte Ltd, 2018.